

逆境之石 报告

目录

- 游戏设计（玩法改动内容、设计思路和目的）
- 程序实现（项目有哪些功能，各功能的实现方法、实现结果，遇到的问题和解决方案）
- 参考资料（列出项目制作中使用到的非自己制作的资源，如教程、代码、美术素材）

一、游戏设计

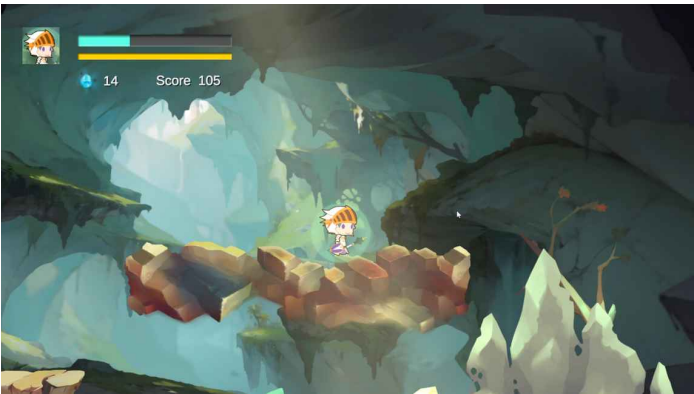
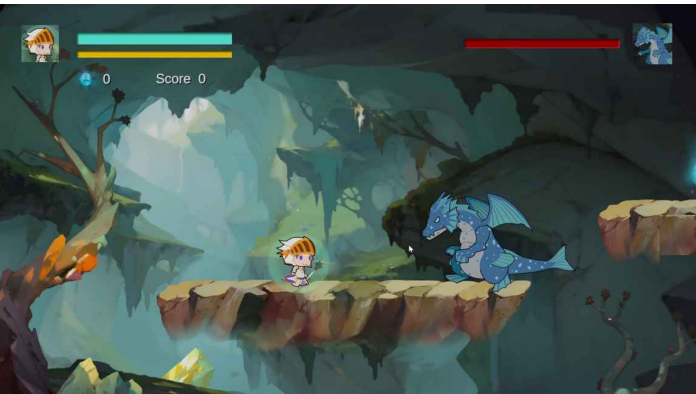
背景

在一个神秘的古老世界中，传说有一种名为“逆时间之石”的神奇物质，据说它能够让人穿越时空，重返过去或者预见未来。然而，逆时间之石在这个世界里异常稀有，只有拥有特殊力量的勇者才能寻得。

主角是一个年轻的冒险家，他听闻了逆时间之石的传说后，决定踏上寻找传说中的水晶之旅。在旅途中，他将面临各种挑战和危险，需要利用逆时间之石的力量解开谜题、避开陷阱，并与龙进行战斗。最终，他是否能够找到逆时间之石，完全取决于玩家的选择和操作。

氛围营造

- 游戏以古老神秘的世界为背景，视觉呈现采用原始风格与奇幻元素相结合，打造神秘感十足的游戏氛围。



玩法以及改动内容

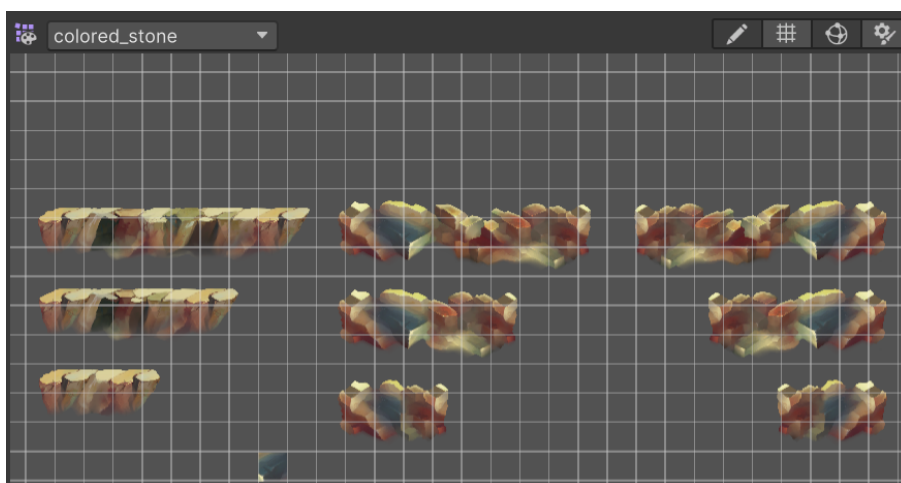
- 灵活移动跳跃，确保不掉落悬崖
- 利用走位和技能击败敌人
- 采集晶石获取分数

- 利用箱子辅助跳跃
- 利用道具渡过河流、采集晶石
- 判断复杂地形，以免坠崖

改动内容：时间原因，未实现晶石暂停时间的功能。

二、程序实现

1. 瓦片地图绘制以及绑定composite collider



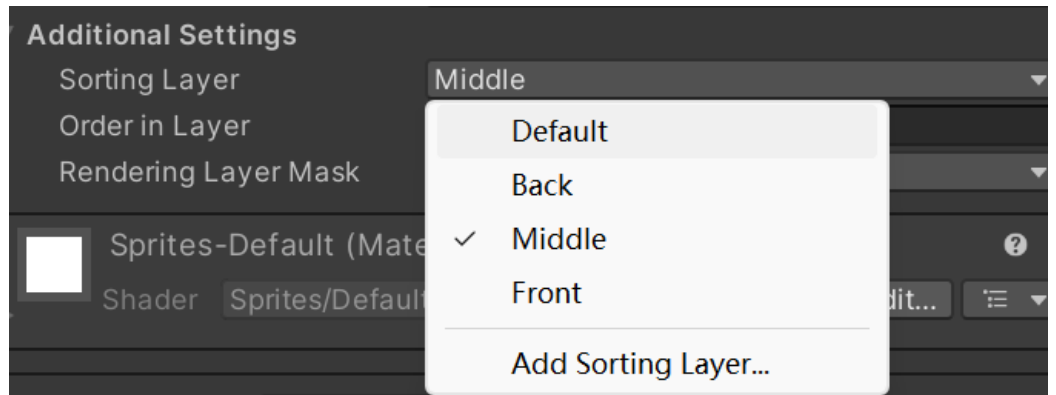
1. **选择瓦片地图：**在Tilemap Palette窗口中使用Active Tilemap下拉菜单选择要编辑的瓦片地图层。
2. **绘制瓦片：**使用画笔工具(B)从Tile Palette中选择瓦片，并在场景视图的瓦片地图上点击或拖动鼠标来绘制瓦片。可以通过按住Shift切换到擦除工具(D)删除瓦片，或使用灌填工具(G)填充区域。
3. **移动和选择瓦片：**使用选择工具(S)选择场景中的瓦片，并用移动工具(M)调整它们的位置。
4. **工具快捷键：**shift+选中为擦除快捷键。
5. **专注特定层：**使用Tilemap Focus功能聚焦于特定的瓦片地图层或整个网格，以免错层。

添加collider：

1. 添加Tilemap Collider 2D
2. 选择Used By Composite，添加Composite Collider 2D，2D 复合碰撞体 (Composite Collider 2D) 能够减少碰撞检测提高性能。



2. 场景叠层设计



3. 输入系统

输入系统由两部分组成

- Gameplay: 关卡进行时控制角色移动、攻击，绑定了KeyBoard和Gamepad两种模式
- UI: 控制UI，例如退出游戏。

4. 角色控制及动画

场景加载事件

- `OnSceneLoad` : 在场景加载时重置角色状态，启动动画，并且暂时禁用游戏操作输入。
- `OnAfterSceneLoadedEvent` : 场景加载完成后重新启动游戏操作输入。

每帧更新（Update方法）

- 读取移动输入：通过 `inputController.Gameplay.Move` 获取玩家的移动输入，并存储到 `inputDirection` 变量中。
- 检查状态：调用 `CheckState` 方法来更新物理材料状态，基于是否在地面上。

物理更新（FixedUpdate方法）

- 移动控制：如果角色没有受伤，调用 `Move` 方法移动角色。移动是通过修改 `Rigidbody2D` 的速度实现的。

移动和翻转（Move方法）

- 实现水平移动：通过修改 `Rigidbody2D` 的速度，结合输入方向和预设速度，来控制角色的水平移动。
- 角色翻转：根据移动方向翻转角色的Sprite，实现视觉上的朝向变化。

跳跃（Jump方法）

- 检查是否在地面上：只有当角色站在地面上时，按下跳跃键才会让角色跳跃。
- 应用跳跃力：通过向上添加一次冲力（Impulse），使角色跳跃。

攻击（Attack方法）

- 触发攻击动画：调用 `PlayerAnimation` 组件的 `Attack` 方法，开始攻击动画。

受伤与死亡（getHurt/die方法）

- 受伤 `getHurt`：角色受到攻击时，速度清零并添加一个方向力使其后退，同时标记为受伤状态。
- 死亡 `die`：角色死亡时，隐藏角色的Sprite，展示游戏结束面板，禁用游戏操作输入，并标记为死亡状态。

碰撞检测（OnTriggerEnter2D/OnCollisionEnter2D方法）

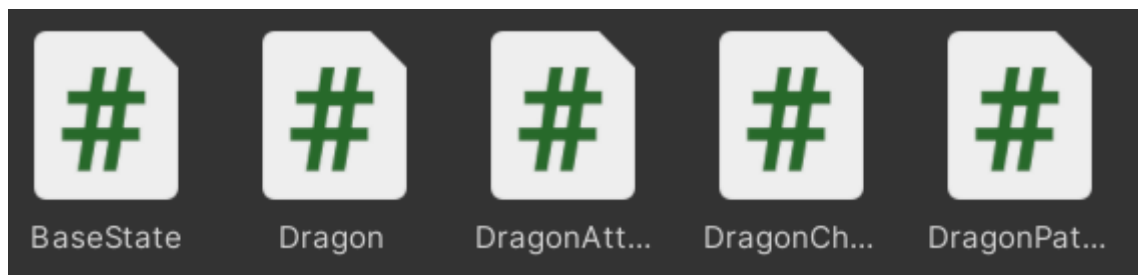
- 检测触发器碰撞：通过标签来确定角色是否碰撞到了特定的触发器（如死亡线、传送门、水等），并执行相应逻辑（如死亡、拾取、恢复）。
- 检测碰撞体碰撞：如角色碰到尖刺，则执行死亡逻辑。

状态检查（CheckState方法）

- 更新物理材质：根据角色是否在地面上来切换使用的物理材质，以改变摩擦等物理属性。

5. 敌人状态机及动画

```
public enum NPCState
{
    Patrol, Chase, Attack
}
```



龙状态机包括三种状态：巡逻（Patrol）、追击（Chase）和攻击（Attack）。每种状态都由一个类表示，且这些类均继承自抽象基类 `BaseState`。状态切换由 `Enemy` 类中的 `SwitchState` 方法控制。

状态切换逻辑

巡逻状态（DragonPatrolState）

- 进入状态时，隐藏敌人的UI，并设定速度为正常速度。
- 在逻辑更新（LogicUpdate）时，检查是否应该攻击玩家，若是，则切换至攻击状态。
- 若未在攻击玩家，但需要追击玩家，则切换至追击状态。

- 如果检测到前方或后方没有地面，则龙将停止并等待（可能会实现翻转等功能，但代码示例中尚未实现）。

追击状态（DragonChaseState）

- 进入状态时，激活敌人的UI，设置速度为追击速度并播放跑步动画。
- 在逻辑更新（LogicUpdate）时，若龙可以攻击玩家，则切换至攻击状态。
- 如果无法追击玩家，则切换回巡逻状态。
- 如果面向的方向没有地面，则翻转龙的方向。

攻击状态（DragonAttackState）

- 进入状态时，激活敌人的UI，将速度设为0，并播放攻击动画。
- 在逻辑更新（LogicUpdate）时，如果龙不再攻击玩家，但可以追击玩家，则切换到追击状态。
- 如果既不攻击也不追击玩家，则切换回巡逻状态。

实现细节

- 每个状态类都必须实现 `BaseState` 中的抽象方法：`onEnter`、`LogicUpdate`、`PhysicsUpdate` 和 `onExit`。
- `onEnter` 方法负责初始化状态（如设置速度和动画）。
- `LogicUpdate` 方法包含状态转换的逻辑。
- `PhysicsUpdate` 方法提供了物理运算的更新点，当前示例中未使用。
- `onExit` 方法负责在退出状态时清理（如停止动画）。

状态机的工作原理建立在 `Enemy` 类中。`Enemy` 类需要包含以下功能：

- 一个当前状态的变量。
- `SwitchState` 方法，负责切换当前状态。
- `isChasePlayer` 和 `isAttackPlayer` 方法，决定是否切换状态。
- `LogicUpdate` 和 `PhysicsUpdate` 方法，分别在每帧调用对应状态的逻辑更新和物理更新方法。

6. 事件创建与使用

事件是一种用来在系统中传递消息的机制。以 `SceneLoadEventSO` 为例。

1. 创建 `ScriptableObject`

`ScriptableObject` 是Unity中用来保存数据的一个类，它不需要附加到 `GameObject` 上。`SceneLoadEventSO` 类继承自 `ScriptableObject`。

2. 定义事件

在类中，我们定义了一个 `UnityAction`，这是Unity中的一个事件委托。`UnityAction` 可以用来监听事件，并在事件发生时调用绑定的方法。

```
1 public UnityAction<GameSceneSO, Vector3, bool> LoadRequestEvent;
```

这个 `UnityAction` 有三个参数：

- `GameSceneSO`：一个自定义的 `ScriptableObject`，用来表示要加载的场景。
- `Vector3`：场景加载后玩家应出现的位置。
- `bool`：表示是否在加载过程中显示淡入淡出效果。

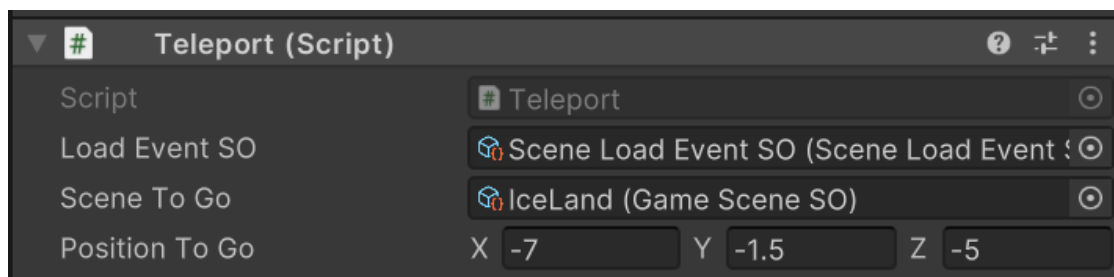
3. 触发事件

我们在 `SceneLoadEventSO` 中定义了一个方法 `RaiseLoadRequestEvent`，用来触发 `LoadRequestEvent` 事件。当游戏想要请求一个场景加载时，它会调用这个方法。

```
1 public void RaiseLoadRequestEvent(GameSceneSO scene, Vector3 position, bool
  fadeScreen)
2 {
3     LoadRequestEvent?.Invoke(scene, position, fadeScreen);
4 }
```

这个方法检查 `LoadRequestEvent` 是否有任何订阅者（即是否为 `null`），如果有，就使用 `Invoke` 方法调用它，传递正确的参数给所有监听者。

7. 场景传送



场景传送系统通过 `Teleport` 和 `SceneManager` 两个脚本实现

Teleport 脚本

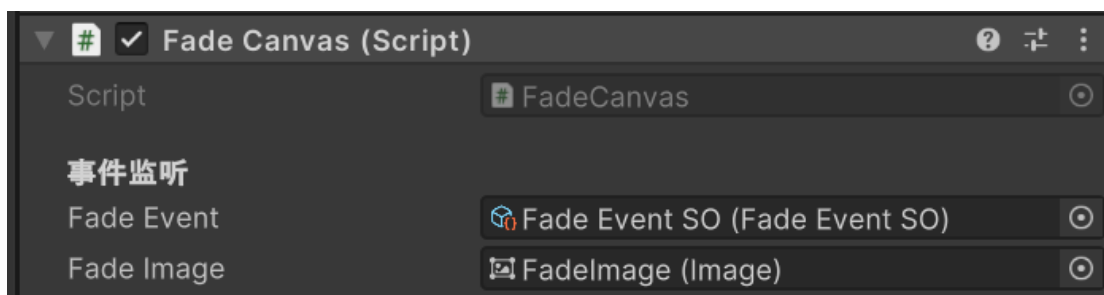
`Teleport` 脚本实现了 `IInteractable` 接口，用于处理玩家与传送点交互时的行为。当玩家触发传送点，通过 `loadEventSO.RaiseLoadRequestEvent` 方法请求加载新场景，同时传递目标场景和位置以及是否使用淡入淡出效果。

SceneManager 脚本

SceneManager 脚本管理场景加载和卸载，以及玩家的位置更新。主要逻辑如下：

1. 在 Awake 中，加载菜单场景，并提出加载初始场景的请求。
2. OnEnable 和 OnDisable 方法分别用于订阅和取消订阅场景加载事件。
3. OnLoadRequestEvent 在收到加载请求时触发，如果当前没有正在加载的场景，它将设置加载标志，并开始卸载当前场景的协程。
4. UnloadPreviousScene 协程实现了可选的淡入效果，等待淡入完成后卸载当前场景，并禁用玩家角色。
5. LoadNewScene 方法启动新场景的异步加载。
6. OnLoadCompleted 是加载完成的回调函数，它更新当前场景引用、设置玩家位置和激活状态。如果使用了淡出效果，它会启动淡出。加载完成后，还会根据场景类型触发后续事件，并更新 Cinemachine 虚拟摄像头的目标。

8. 场景渐隐渐显



场景渐隐渐显功能的实现主要依赖于 FadeCanvas 和 SceneManager 两个脚本。

1. 广播渐隐事件

当需要加载新的场景时，首先触发一个渐隐事件，将当前画面逐渐变为全黑或目标颜色：

```
1 if (isFade)
2 {
3     fadeEvent.FadeIn(fadeDuration);
4 }
```

2. 执行渐隐动画

在 FadeCanvas 脚本中，监听到渐隐事件后，使用 DOTween 插件执行颜色渐变动画：

```
1 \private void OnFadeEvent(Color target, float duration, bool fadeIn)
2 {
```

```
3     fadeImage.DOBlendableColor(target, duration);
4 }
```

3. 卸载当前场景

当前场面完全变黑后，等待渐隐动画完成（使用 `fadeDuration`），然后开始卸载当前场景的操作：

```
1 yield return new WaitForSeconds(fadeDuration);
2 currentScene.sceneReference.UnLoadScene();
```

4. 加载新场景

在卸载当前场景之后，启动新场景的加载：

```
1 private void LoadNewScene()
2 {var loadNewScene =
    sceneToLoad.sceneReference.LoadSceneAsync(LoadSceneMode.Additive, true);
3     loadNewScene.Completed += OnLoadCompleted;
4 }
```

5. 新场景加载完成

一旦新场景加载完成，将玩家的位置设置到目的地，并重新激活玩家游戏对象：

```
1 private void OnLoadCompleted(AsyncOperationHandle<SceneInstance> handle)
2 {
3     playerTransform.position = posToGo;
4     playerTransform.gameObject.SetActive(true);
5 }
```

6. 广播渐显事件

完成场景切换后，如果需要渐显，广播渐显事件，使画面由全黑逐渐恢复到正常：

```
1 if (isFade)
2 {
3     fadeEvent.FadeOut(fadeDuration);
4 }
```

7. 执行渐显动画

和渐隐类似，`FadeCanvas` 会监听到渐显事件，并使用DOTween插件执行相应的动画。

9. 音频混合

`AudioManager` 是 Unity 中用于管理音频播放的组件，用于处理背景音乐（BGM）、音效（FX）和敌人相关音效的播放。

组件属性

- `BGMEvent` : 负责背景音乐播放的事件。该属性在代码中未使用，可能在其他地方触发事件。
- `FXEvent` : 负责音效播放的事件。
- `EnemyEvent` : 负责敌人音效播放的事件。
- `BGMSource` : 背景音乐的音频源。
- `FXSource` : 音效的音频源。
- `EnemySource` : 敌人音效的音频源。

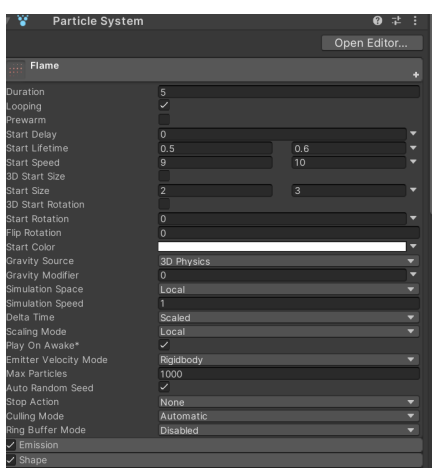
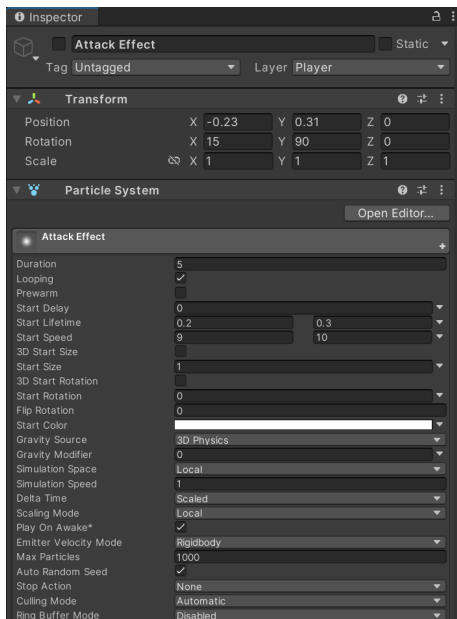
事件处理

- `OnFXEvent(AudioClip clip)` : 当音效事件被触发时，此方法被调用，并接收一个 `AudioClip` 参数。
 - 方法将接收到的 `AudioClip` 设置为 `FXSource` 的剪辑，并播放该剪辑。
- `OnEnemyEvent(AudioClip clip)` : 当敌人音效事件被触发时，此方法同样被调用，并接收一个 `AudioClip` 参数。
 - 方法将接收到的 `AudioClip` 设置为 `EnemySource` 的剪辑，并播放该剪辑。

10. 粒子特效

粒子系统实现

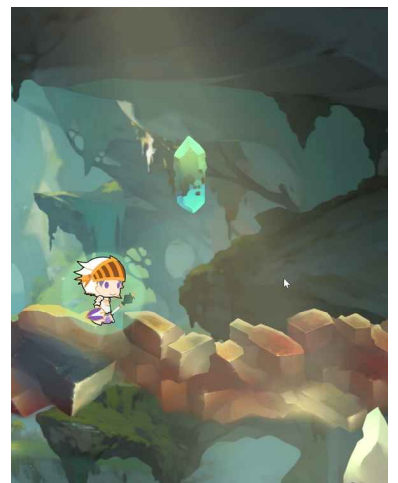
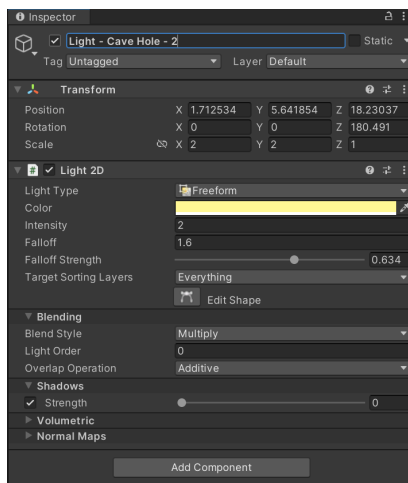
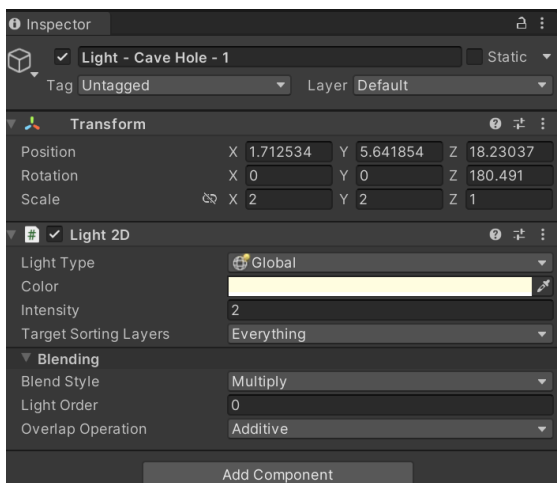
- 龙喷火
- 角色魔法攻击



11. 2D光照

基于Light 2D实现

- 全局光照
- 局部锥形光照，模拟丁达尔效应。



12. UI控制

事件监听

- `healthEvent` : 角色血量发生变化时触发的事件。
- `scoreManagerEvent` : 分数管理器事件，用于响应分数变化事件。

血量更新实现

- `OnHealthEvent` : 响应血量变化事件的方法。
 - 根据角色对象标签 ("Player"或"Enemy") 以及敌人的名称 ("Dragon1"或"Dragon2")，决定调用 `playerUI.bloodUpdate` 或 `enemyUI.bloodUpdate` 方法。
 - 使用角色的当前血量与最大血量的比例来更新对应的血量条UI。

分数和水晶数量更新实现

- `OnScoreManagerEvent` : 响应分数管理器事件的方法。
 - 更新水晶数量显示 `crystalText` 的文本为分数管理器中记录的水晶数量 `crystalNum`。
 - 更新分数显示 `scoreText` 的文本为"Score "加上分数管理器中记录的总分数 `totalScore`。

13. bug和对应解决方案

13.1 添加Animator后人物移动上升问题

- **问题描述**：龙添加animator后，会缓慢移动上升，经排查问题出在animator上，并非rigidbody中gravity设置为负向。
- **解决方案**：关闭Apply Root Motion后即可解决。

13.2 死亡动画结尾部分反复播放

- **问题描述**：animator中，`any state` 经过 `isDead = true` 的判断条件后，会转移到 `Dead` 状态播放死亡动画。`Dead` 状态后会经过 `solo` 的transition进入 `Exit` 状态，我只希望播放一遍死亡动画就停止。
- **解决方案**：`any state` 到 `Dead` 的transition设置为 `can transition to self = false`，这是 `any state` 下一个状态的特殊参数。

13.3 Cinemachine与视差滚动功能一起使用会抖动

- **问题描述**：本项目中使用cinemachine插件实现人物跟随，同时绑定parrallex background脚本更新背景的移动实现视差滚动，游戏运行时画面一直抖动。

- **解决方案：**ParallaxScrollingFarX脚本的Update时机与Cinemachine Brain脚本的Update时机不同，需要找出 Cinemachine Brain 脚本的Update时机。在脚本被激活时，添加对 CameraUpdatedEvent事件的监听即可。代码如下

```
1 private void OnEnable()  
2 {  
3     CinemachineCore.CameraUpdatedEvent.AddListener(CameraUpdate);  
4 }  
5  
6 private void OnDisable()  
7 {  
8     CinemachineCore.CameraUpdatedEvent.RemoveListener(CameraUpdate);  
9 }  
10  
11 private void CameraUpdate(CinemachineBrain arg0)  
12 {  
13     UpdateParallaxScrolling();  
14 }
```

13.4 脚本新增public变量更新后未在inspector中暴露

- **问题描述：**碰到过好几次，脚本新增public变量更新后，inspector面板中对应脚本并不显示新增变量
- **解决方案：**经排查，一旦有script编译失败，就会导致public变量不及时显示，特地记一笔，以防以后忘记。

三、参考资料

1. 教程：M_Studio 《勇士传说》 Unity2022版教程
<https://space.bilibili.com/370283072/channel/collectiondetail?sid=1187255>
2. 美术&音频素材，插件：

Action RPG Music Free	1.6	
Animated Pixel-Art Backgrounds Free	1.1	
Colourful Hierarchy Category GameObject	1.2	
DOTween (HOTween v2)	1.2.765	
Footsteps - Essentials	1.3	
[FreeBGM] Breezy Hill, Sunset Drive	1.0	
FREE Casual Game SFX Pack	1.0	
Free Casual Music Pack	1.0	
Lost Crypt - 2D Sample Project	1.3.2	
Sci-fi GUI skin	1.2	
Stone UI	1.0	